Software Requirements Specification (SRS)

College Help Desk Agent

---

1. Introduction

1.1 Purpose

The purpose of this document is to define the functional and non-functional requirements of the **College Help Desk Agent**, a web-based application designed to assist students in retrieving college-related information quickly and accurately.

1.2 Scope

The system is a **single-page web application** that provides:

- A chatbot for answering student queries using a Retrieval-Augmented Generation (RAG) approach (data retrieval only)
- Structured information sections within the homepage

The system does **not include**:

- User authentication
- Admin panel

1.3 Definitions

- **RAG (Retrieval-Augmented Generation):** System retrieves relevant data from a knowledge base and returns it as a response (no generative AI content).
- **Knowledge Base:** Structured college data stored in JSON/database.

---

2. Overall Description

2.1 Product Perspective

The application is a **standalone full-stack web system** consisting of:

- Frontend (ReactJS)
- Backend (Node.js + Express)
- AI Retrieval Layer (LangChain + Google GenAI)

2.2 Product Functions

The system will:

- Accept natural language queries from users
- Retrieve relevant data from structured knowledge base
- Display responses conversationally
- Show structured college information in UI components

2.3 User Characteristics

- Target Users: Students
- Skill Level: Basic web users
- No login required

2.4 Constraints

- No authentication system
- No admin panel
- Data is static (JSON/database)
- AI is limited to retrieval, not generation

2.5 Assumptions

- Knowledge base contains accurate and structured data
- Users have internet access
- Browser supports modern JavaScript

---

3. System Features

3.1 Chatbot System

Description

Allows users to ask questions in natural language and receive accurate responses from the knowledge base.

Functional Requirements

- User can type and send a message
- System processes input query
- Backend retrieves relevant data
- System returns a clear response
- Chat history is displayed in UI

Inputs

- Text query from user

Outputs

- Retrieved response from knowledge base

3.2 Homepage (Single Page Layout)

Description

All sections are displayed as components within one page.

Sections Included

- Hero / Introduction Section
- Chatbot Section
- Courses Information Section
- Admissions / Fees Section
- Placement Information Section

3.3 Courses Section

Description

Displays list of available programs.

Functional Requirements

- Show course name

- Show description/details
- Responsive layout

3.4 Admissions & Fees Section

Description

Displays admission process and fee details.

Functional Requirements

- Show eligibility criteria
- Show fee structure
- Organized and readable format

3.5 Placement Section

Description

Displays placement statistics and insights.

Functional Requirements

- Show placement percentage
- Show top recruiters (if available)
- Show average/highest packages

3.6 Navigation

Description

Smooth scrolling navigation within homepage.

Functional Requirements

- Scroll to sections on click
- Responsive navigation menu

---

4. External Interface Requirements

4.1 User Interface

- Clean and minimal design
- Responsive (mobile + desktop)
- Chat UI similar to messaging apps

4.2 Software Interface

- Frontend communicates with backend via REST APIs
- API communication via Axios/Fetch

4.3 Hardware Interface

- Runs on standard web browsers
- No special hardware required

---

5. Non-Functional Requirements

5.1 Performance

- Response time < 3 seconds for chatbot queries

5.2 Usability

- Beginner-friendly interface
- Easy navigation
- Clear content presentation

5.3 Reliability

- Handles empty input gracefully
- Handles API failures with error messages

5.4 Scalability

- System should support adding more data in knowledge base

5.5 Maintainability

- Modular code structure
- Separate frontend and backend layers

5.6 Security

- Basic input validation
- No sensitive data handling (no login)

---

6. System Architecture

Architecture Style

- Client-Server Architecture

Components

- React Frontend
- Express Backend API
- Knowledge Base (JSON/DB)
- Retrieval Layer (LangChain)

---

7. Data Requirements

Data Storage

- Structured JSON or database

Data Types

- Courses
- Fees
- Admissions info
- Placement data

---

8. Error Handling

- Empty input → Show validation message
- API failure → Show error notification
- No data found → Show fallback response

---

9. Future Enhancements (Optional)

- Add authentication system
- Add admin panel
- Add real AI-generated responses
- Multi-college support